宁波大学

本科实验报告

姓名：	梁航锋
学院：	信息科学与工程学院
系：	计算机科学与技术系
专业：	计算机科学与技术
学号：	216001912
指导教师：	董理

2024 年 3 月 14 日

宁波大学实验报告

课程名称：机器学习 实验类型：验证性

实验项目名称：实验二：线性回归与逻辑回归

学生姓名：梁航锋 专业：计算机科学与技术 学号：216001912

同组学生姓名：

指导老师：董理

实验地点：计算机组成实验室 201 实验日期：2024 年 3 月 27 日

一、实验目的和要求

1. 学习掌握相关理论知识，包括线性回归、逻辑回归的基本原理，模型训练方法等，并通过编程，从实践的角度深化模型的理解；

2. 学习使用梯度下降法进行模型训练的基本原理，并自行使用代码进行梯度下降的实现，将其运用到线性回归、逻辑回归模型的训练；

3. 理解学习率、迭代次数等参数的含义，并通过实验探究其对模型训练过程产生的影响；

4. 通过绘图进行训练过程、模型预测效果的可视化，并通过均方误差等指标对模型的性能进行衡量。

二、实验内容和原理

1. 使用 sklearn 框架实现线性回归与逻辑回归，模型的训练与测试基于给定的西瓜数据；

2. 基于梯度下降法的原理进行代码实现，并将其运用于步骤 1 的实验内容中，探究学习率、迭代次数对模型训练过程的影响；

3. 完成实验报告的攥写，并及时提交实验报告。

三、实验环境

操作系统：

Windows 11 Version 22H2

集成开发环境：

1. Anaconda3 with Jupyter Notebook
2. PyCharm 2022.2.2 (Professional Edition)

相关 Python 库：

1. python3.9
2. numpy
3. matplotlib（3.8.3）
4. sklearn（scikit-learn1.4.1）
5. pandas（2.2.1）

四、操作方法与实验步骤

1. 实验准备

在第一次实验中已经完成了 conda 虚拟环境搭建，故本次实验基于该虚拟环境进行。对于实验所用数据，使用实验指导给定的西瓜数据，分别用于回归问题和分类问题。数据内容如图 4.1 所示。

编号	密度	含糖率	好瓜
1	0.697	0.46	1
2	0.774	0.376	1
3	0.634	0.264	1
4	0.608	0.318	1
5	0.556	0.215	1
6	0.403	0.237	1
7	0.481	0.149	1
8	0.437	0.211	1
9	0.666	0.091	0
10	0.243	0.0267	0
11	0.245	0.057	0
12	0.343	0.099	0
13	0.639	0.161	0
14	0.657	0.198	0
15	0.36	0.37	0
16	0.593	0.042	0
17	0.719	0.103	0

编号	密度	含糖量	成熟度
1	0.697	0.46	0.89
2	0.774	0.376	0.9
3	0.634	0.264	0.76
4	0.608	0.318	0.78
5	0.556	0.215	0.58
6	0.403	0.237	0.61
7	0.481	0.149	0.6
8	0.666	0.091	0.48
9	0.243	0.268	0.42
10	0.245	0.057	0.33
11	0.343	0.099	0.37
12	0.639	0.161	0.45
13	0.657	0.198	0.4
14	0.36	0.37	0.22

图 4.1 实验数据（左-回归问题，右-分类问题）

2. 使用 sklearn 库实现模型训练

（1）数据预处理

给定的数据存于 excel 表格当中，使用 pandas 工具进行读入，并将数据整合成标准形状的 numpy 数组，便于后续的模型训练，其中数据特征的形状应当为[n,2]，数据标签应为[n,1]，具体的处理方法详见代码。

（2）训练、测试集划分

使用 train_test_split()函数进行数据集的划分，该函数进行数据集的随机划分且保证划分结果互斥。其中设定测试集的数据占比 10%。

（3）模型训练

对于 sklearn 库，已经完成了基本的模型训练过程封装，只需要进行模型的实例化，调用 fit()函数传入训练数据即可完成训练，调用 predict 函数并传入测试数据的特征即可得到对应的结果。

其中线性回归模型为 LinearRegression()，分类问题对应的逻辑回归模型为 LogisticRegression()。

（4）训练结果查看

为查看模型的训练效果，对训练数据的原始标签与模型输出标签进行绘图呈现。数值方面，使用均方误差这一指标进行衡量。同时，也可以访问对象的 coef_属性查看模型对应的参数权重。

2. 使用 sklearn 库实现模型训练

（1）损失函数定义

机器学习模型训练的本质是通过调整模型参数使得模型输出结果与真实结果之间的差异最小，而衡量差异通常使用损失函数（Loss Function），对于回归问题，损失函数通常为均方误差，即所有样本数据的真实标签与模型输出标签之间的差值平方之和，表达式如下：

$$Loss(\theta)=\frac{1}{2m}\sum_{i=1}^{m}(y^{(i)}-f_\theta(x^{(i)}))^2$$

对于逻辑回归模型，用于二分类问题，其使用对数损失函数，也称为交叉熵损失函数，表达式如下，其中模型 f 的输出经过 sigmoid 函数进行约束。

$$Loss(\theta)=-\sum_{i=1}^{m}y^i\log(f_\theta(x^i)+(1-y^i)\log(1-f_\theta(x^{(i)})$$

使用 Python 代码分别实现以上损失函数。

（2）定义训练函数

进行训练函数的定义，该函数使用梯度下降法进行模型的训练，函数内需要进行多轮迭代，每次迭代中，需要以模型参数为自变量，对损失函数进行求

导，得到当前时刻的梯度值，该梯度值指示了损失函数变化最快的方向，将模型参数往该方向进行更新，更新的步长由学习率进行约束。形式化地表达如下：对于每个参数θ_j，调整过程如下式所示，其中 α 为学习率：

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} Loss(\theta)$$

具体地，该函数需要传入以下内容：模型参数、训练集特征、训练集标签、迭代次数、学习率，通过函数内的迭代，模型参数不断调整，过程中可以记录损失函数以查看模型的训练过程。

（3）模型的训练与验证

该步骤与使用 sklearn 库进行模型训练验证的步骤一致，只是将模型的训练转移到了自行编写的基于梯度下降的训练函数中。最后需要进行训练结果的查看与验证。

3. 学习率调整

对模型的学习率与迭代次数进行调整，记录损失函数的变化过程，分析学习率与迭代次数对模型训练的具体影响。

五、实验数据记录和处理

该部分将按照代码逻辑与以上实验步骤进行代码的划分，并记录。其中代码部分中，通过 sklearn 实现线性回归与逻辑回归的代码高度相似，为节约篇幅不进行赘述。

1. 线性回归与逻辑回归

```
# 相关库导入
import LinearRegressionimport matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from sklearn import model_selection
from sklearn import metrics
from sklearn import linear_model
from tqdm import tqdm
```

```
# 线性归回数据读入
data_path = 'Linear Dataset.xlsx'
data = pd.read_excel(data_path)
```

```
x_0 = data['密度'].to_numpy().reshape(-1,1)  # shape 为[n,1]
x_1 = data['含糖量'].to_numpy().reshape(-1,1)
y = data['成熟度'].to_numpy().reshape(-1,1)
x = np.hstack((x_0,x_1))  # shape 为[n,1]
```

```
# 模型训练
x_train,x_test,y_train,y_test=model_selection.train_test_split(x,y,te
st_size=0.1) # 数据集划分
linear = linear_model.LinearRegression()  # 模型实例化
# logistic = linear_model.LogisticRegression()  逻辑回归
linear.fit(x_train, y_train) # 模型训练（拟合）
```

```
# 绘制图像：显示模型输出与真实结果，观察拟合程度
y_train_pre = linear.predict(x_train)
plt.plot([i for i in range(len(y_train_pre))], y_train_pre)
plt.plot([i for i in range(len(y_train))], y_train)
plt.legend(['model label','true label'], loc='best')
plt.xlabel('id')
plt.ylabel('value')
plt.title('Train result')
plt.show()
```

```
# 测试集上的均方误差与模型参数
y_test_pre=linear.predict(x_test)
SQ_ERR = metrics.mean_squared_error(y_test_pre, y_test)
print(SQ_ERR, linear.coef_)
```

2. 梯度下降的实现

该部分使用梯度下降法进行线性回归模型与逻辑回归模型训练。

线性回归训练代码：

```
def LossFunction(theta,feature,label):   # 均方误差
    return np.sum((np.dot(feature,theta)-
                                label)**2)/(2*feature.shape[0])
# 线性回归模型训练
def linear_train(theta,feature,label,iter=100,lr=0.001,loss_list=None
):
    num=feature.shape[0]
    for i in tqdm(range(iter)):
        # 梯度计算
```

```
        dL=np.dot(feature.transpose(),(np.dot(feature,theta)-
                                                label))/num
        theta=theta-lr*dL # 沿梯度下降
        Loss=LossFunction(theta,feature,label)
        loss_list.append(Loss)
# 根据模型参数进行预测
def predict(theta, feature):
    num=feature.shape[0]
    pre=[]
    for i in range(num):
        pre.append(np.dot(feature[i],theta))
    return np.array(pre)
```

逻辑回归训练代码：

```
# sigmoid 函数，值域(0,1)
def sigmoid(z):
    return 1/(1+np.exp(-z))
# 逻辑回归损失函数（交叉熵）
def LossFunction(theta,feature,label):
    num=feature.shape[0]
    return -
1/num*(np.dot(label.transpose(),np.log(sigmoid(np.dot(feature,theta))
))+np.dot((1-label).transpose(),np.log(1-
sigmoid(np.dot(feature,theta)))))
# 逻辑回归模型训练
def logistic_train(theta,feature,label,iter=100,lr=0.001,loss_list=No
ne):
    num=feature.shape[0]
    for i in tqdm(range(iter)):
        # 梯度计算
        dL=np.dot(feature.transpose(),(sigmoid(np.dot(feature,theta))
                                            -label))/num
        theta=theta-lr*dL  #沿着梯度下降
        Loss=LossFunction(theta,feature,label).item()
        loss_list.append(Loss)
# 根据模型参数进行预测
def predict(theta, feature):
    pre=np.zeros([feature.shape[0],1])
    for i,val in enumerate(np.dot(feature,theta)):
        if sigmoid(val)>=0.5:
            pre[i]=1
        else:
            pre[i]=0
    return pre
```

六、实验结果与分析

该部分将对实验代码产出的相关结果与图像进行记录与分析。

1. sklearn 线性回归

使用 sklearn 库进行线性回归模型的训练，在训练集上的训练结果如下图 6.1 所示。在测试集上的均方误差为 0.01073，模型所给出的系数比为 [0.6510,0.7336]，由此可以看出含糖量对于最终成熟度的影响最大。

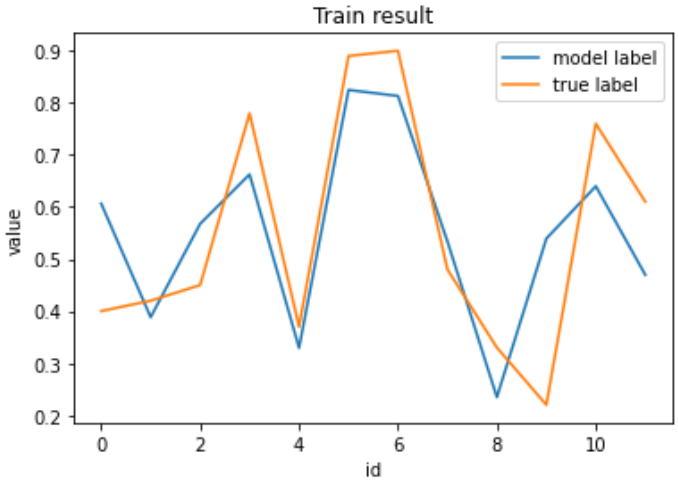


图 6.1 sklearn 库下的线性回归模型拟合效果

2. sklearn 逻辑回归

对于逻辑回归，考虑计算分类的准确率，代码部分如下图 6.2 所示。可以看到模型对于测试集的准确率为 0.5，在训练集上拟合后的准确率为 0.52。

```
In 67  1  y_test_pre = logistic.predict(x_test)
       2  y_test,y_test_pre

Out 67    (array([[1],
                  [0]], dtype=int64),
           array([0, 0], dtype=int64))

In 68  1  acc = np.mean(y_test_pre==y_test)
       2  acc_train = np.mean(y_train_pre==y_train)
       3  acc_train,acc

Out 68    (0.5244444444444445, 0.5)
```

图 6.2 模型拟合与验证（多项式度数=4）

3. 梯度下降法

使用梯度下降法实现模型的训练过程，代码见第 5 节所述，在迭代训练过程中记录 Loss 的变化，以描述训练的过程。

将自定义的训练函数运用于线性回归模型，首先随机初始参数值，设置学习率为 0.0001，迭代次数为 100000，Loss 变化趋势如图 6.3 所示。

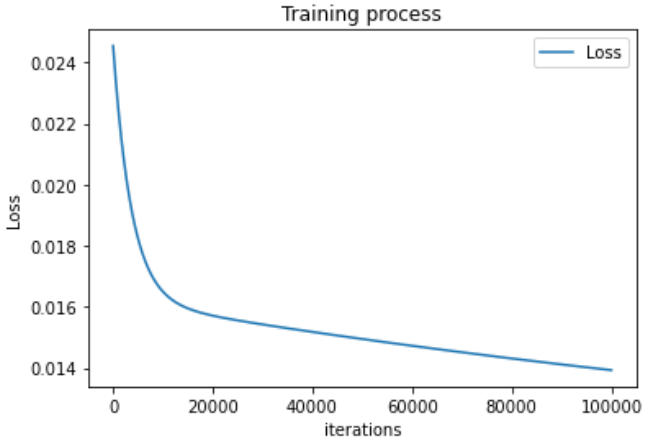


图 6.3 线性回归 Loss 变化趋势（梯度下降法）

数据的拟合结果如图 6.4 所示，模型在测试集上的均方误差为 0.078。

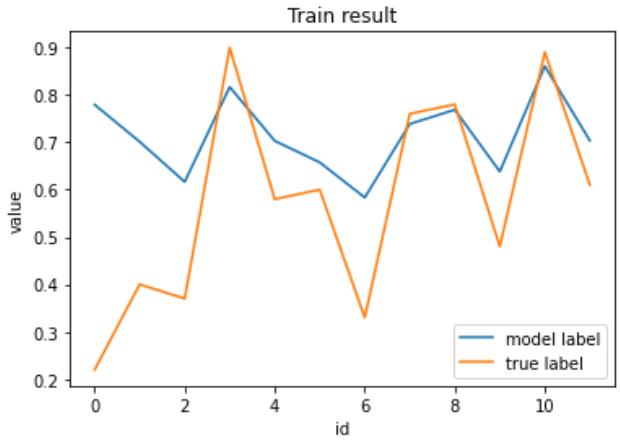


图 6.4 线性回归拟合效果（梯度下降法）

将自定义的训练函数运用于逻辑回归模型，首先随机参数值，设置学习率为 0.0001，迭代次数为 100000，Loss 变化趋势如图 6.5 所示。

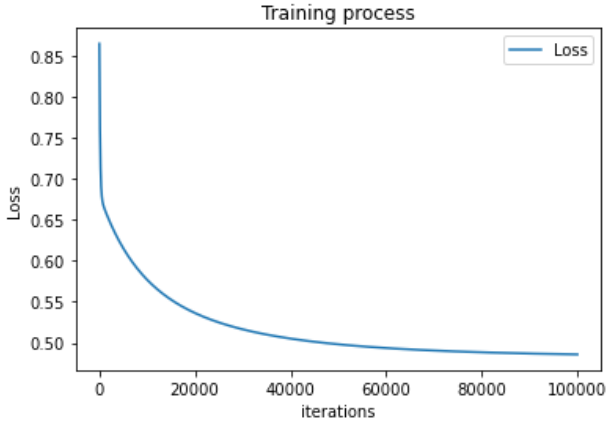


图 6.5 逻辑回归拟合效果（梯度下降法）

模型最终效果与 sklearn 库的效果类似。

4. 学习率的影响

通过理论学习可知，当学习率设置过大时，可能会导致 Loss 跳跃过多，从而错过 Loss 函数和的最低点，而导致模型训练不收敛，如图 6.6 所示，将学习率设置为 10，发现 Loss 最后不减反增。

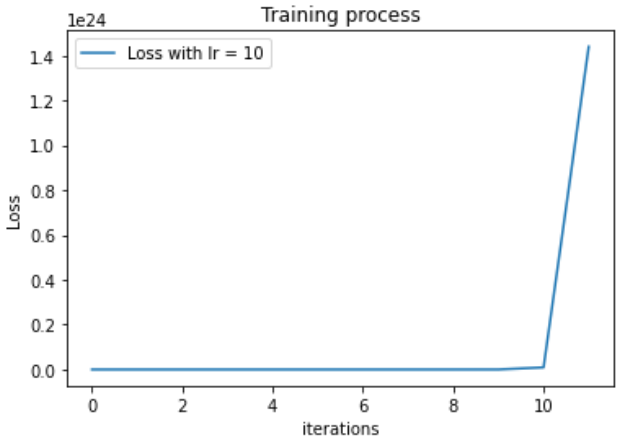


图 6.4 模型拟合与验证（多项式度数=25）

当学习率设置过小时，会减缓模型的收敛速度，若没有足够多的迭代次数，则会导致无法到达较优的参数中，另一方面也增加了模型的训练时长。

七、讨论、心得

本次实验基于 Python 与 sklearn 库进行了基本的线性回归、逻辑回归模型的训练与测试，并在西瓜数据集上进行了分类与预测操作。此外，基于梯度下降思想，本次实验进行了梯度下降法的代码编写，并使用编写的训练函数替换原有的 sklearn 库函数，实现了相同的训练效果。

在梯度下降法中，通常需要进行学习率的设置，该参数作为机器学习模型甚至深度学习模型训练中的重要指标，将直接影响模型的收敛速度与训练效果，通过日常经验、理论分析、实验验证，可以发现对于学习率的设置的困难，有大量的研究投入于对“如何设置好的学习率”之中。当学习率设置过大时，对于某些模型的训练会产生损失不减反增的情况，这是由于过大的学习率导致了过大的步长，从而产生了越过函数较低点的情况；而当学习率设置过小时，则会产生模型收敛速度慢的情况，导致模型训练效果不理想，从而被迫引入较长的学习时长。一种常见的学习率设置方法为随着迭代次数增加而减小学习率。